НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені Ігоря СІКОРСЬКОГО»
Навчально-науковий фізико-технічний інститут
Кафедра математичних методів захисту інформації

Лабораторна робота №1 з дисципліни Методи криптоаналізу II

Роботу виконували:
Юрчук Олексій, ФІ-52мн
Дигас Богдан, ФІ-52мн

27 березня 2026 р.
м. Київ

ЗМІСТ

1 Intro **1**
1.1 Вхідні дані 1
1.2 Хід виконання роботи 1
1.2.1 Побудова базисів Грьобнера 2
1.2.2 Побудова системи рівнянь меншого степеня 3
1.2.3 Розв'язання системи 4
1.2.4 Перевірка 5
1.3 Висновки 6
1.4 Code listing section 6

Список використаних джерел **15**

1

Intro

$$\text{Варіант} = \max\{5; 10\} = 10$$

Мета комп'ютерного практикуму: Практична реалізація алгебраїчної атаки на фільтрувальний генератор гами; набуття навичок роботи з системами комп'ютерної алгебри в SAGE. Гроь
Постановка задачі:

1. Знайти функції мінімального степеня ідеалів $\langle f \rangle$ та $\langle f \oplus 1 \rangle$ за допомогою побудови базису Грьобнера. Якщо побудова базису для одного з ідеалів $\langle f \rangle$ або $\langle f \oplus 1 \rangle$ є занадто трудомісткою з точки зору обчислювальних ресурсів, то будувати лише один базис – за умови, що цього буде достатньо для проведення атаки.

2. Визначити кількість рівнянь, необхідних для відновлення початкового стану (якщо не вдається визначити аналітично, то емпірично шляхом підбирати). Побудувати систему рівнянь меншого степеня відносно початкового стану генератора.

3. Знайти розв'язки отриманої системи рівнянь. Зауважимо, що початковий стан за умовою комп'ютерного практикуму є ненульовим вектором.

4. Перевірити, що початковий стан відновлено правильно, шляхом генерування відрізка гами відповідної довжини й порівнявши його з вхідними даними.

1.1 Вхідні дані

1. Довжина регістру 64 біти. Поліном зворотного зв'язку є наступним:

$$\begin{aligned} p(x) = & x^{64} \oplus x^{63} \oplus x^{62} \oplus x^{60} \oplus x^{59} \oplus x^{58} \oplus x^{57} \oplus x^{56} \oplus x^{53} \oplus x^{50} \oplus x^{47} \oplus x^{45} \oplus x^{44} \oplus x^{43} \oplus \\ & \oplus x^{42} \oplus x^{41} \oplus x^{40} \oplus x^{39} \oplus x^{38} \oplus x^{37} \oplus x^{36} \oplus x^{34} \oplus x^{32} \oplus x^{30} \oplus x^{28} \oplus x^{24} \oplus x^{18} \oplus \\ & \oplus x^{15} \oplus x^{14} \oplus x^{13} \oplus x^{11} \oplus x^{9} \oplus x^{6} \oplus x^{4} \oplus 1. \end{aligned}$$

2. Фільтрувальні функції $f(x_0, \dots, x_{63})$ генератора гами збережені у **filterfuncV10.txt**, $\deg f = 20$, містить 39 мономів.

3. Відрізок гами $\gamma_1, \dots, \gamma_N$ довжини $N = 50000$ бітів у файлі **gammaV10.txt**.

1.2 Хід виконання роботи

Я встановив пакет комп'ютерної алгебри SageMath (версія 10.7) в свій WSL 2 Ubuntu за інструкцією https://doc.sagemath.org/html/en/installation/index.html.

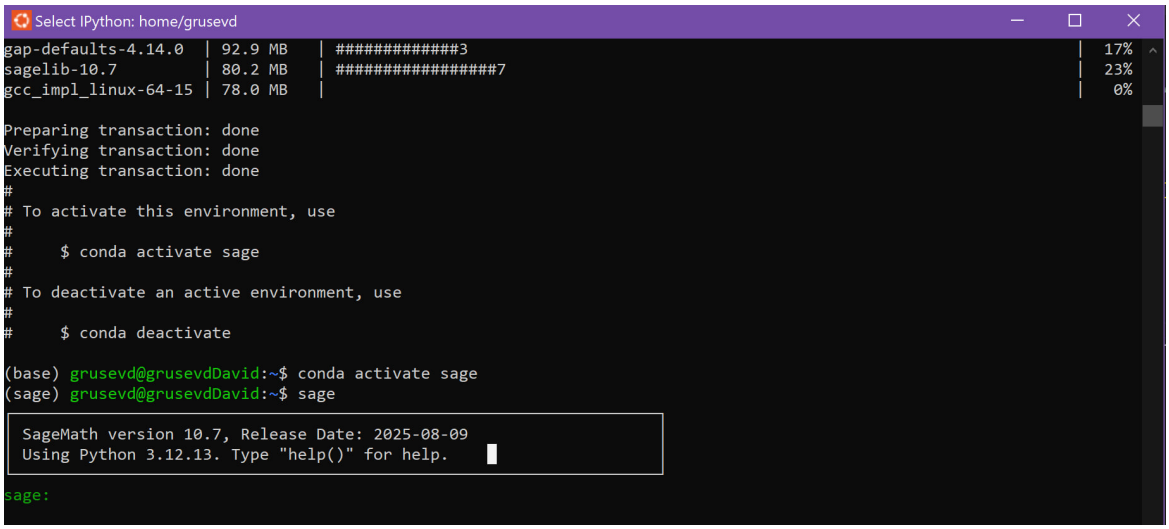


Рис. 1.1: Встановили SAGE

1.2.1 Побудова базисів Грьобнера

Кільце булевих поліномів $B_{64} = \mathbb{F}_2[x_0, \dots, x_{63}]/\langle x_i^2 \oplus x_i \rangle$ ініціалізовано за допомогою мономіального впорядкування $degrevlex$ (степенево-зворотне лексикографічне) – найефективніше для базисів Грьобнера (звичайний lex був би корисним для виключення змінних, і їв при цьому більше часу), яке є степеневим, що гарантує коректність застосування теореми Арса–Фожере (вона обґрунтовує застосування базисів Грьобнера для пошуку функцій найменшого степеня в ідеалі).

Базис ідеалу $\langle f \rangle$. Час побудови: ≈ 578 с. Потужність базису: 10047.

Функції мінімального степеня ідеалу $\langle f \rangle$: степінь $d_1 = 2$, кількість: 1.

Єдина функція мінімального степеня:

$$g_1 = x_{61} \cdot x_{21} \oplus x_{61}.$$

Зауважимо, що $g_1 = x_{61}(x_{21} \oplus 1)$, тобто $g_1 \in \langle f \rangle = \mathrm{Ann}(f \oplus 1)$ і $g_1 \cdot (f \oplus 1) = 0$.

Базис ідеалу $\langle f \oplus 1 \rangle$. Не будувався, точніше як... Побудова $\langle f \oplus 1 \rangle$ впала через нестачу пам'яті (вбудований sage алгоритм PolyBoRi (Polynomials over Boolean Rings) завершився з помилкою `"Built-in matrix-size exceeded!"` через перевищення внутрішнього ліміту на розмір матриці) Це саме та ситуація, яка описувалася в завданні: "будувати лише один базис, за умови, що цього буде достатньо для проведення атаки". Як раз наш випадок :)

Алгебраїчна імунність. Оскільки мінімальний степінь функцій в $\langle f \rangle$ дорівнює 2, а $\mathrm{AI}(f) = \min(d_1, d_2) \leq d_1$, маємо:

$$\mathrm{AI}(f) \leq 2.$$

Для порівняння: оптимальне значення для $n = 64$ (довжина регістру) становить $\lceil 64/2 \rceil = 32$. Таким чином, функція має надзвичайно низьку алгебраїчну імунність і є вразливою до алгебраїчної атаки [1].

1.2.2 Побудова системи рівнянь меншого степеня

Через $e = 2$ позначимо степінь знайденої функції $g_1 \in \langle f \rangle$, що використовується для побудови рівнянь-наслідків. Кількість мономів степеня не більше e у $n = 64$ змінних обчислюється формулою:

$$M = \sum_{k=0}^{2} C_{64}^{k} = 1 + 64 + 2016 = 2081.$$

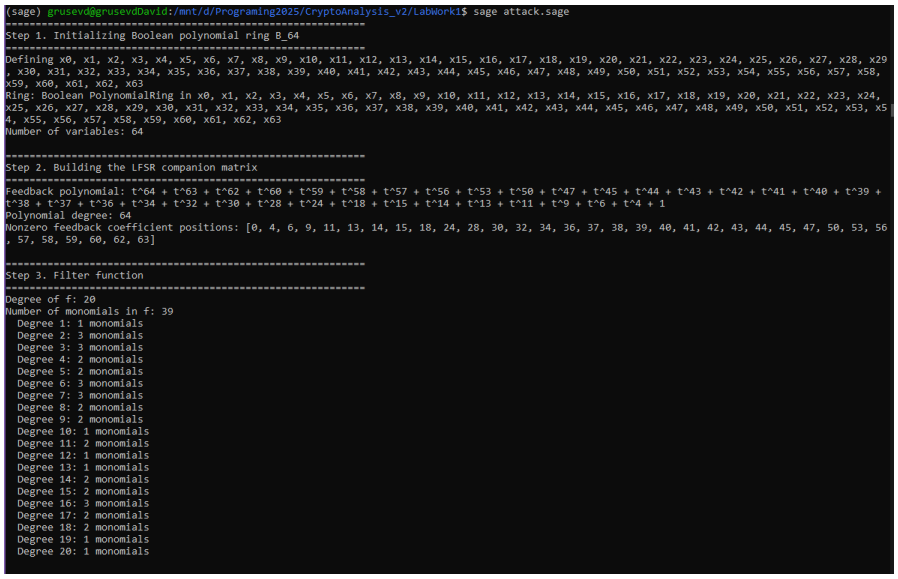
Оскільки побудувати базис $\langle f \oplus 1 \rangle$ не вдалось, то рівняння будуватимемо лише з тактів, де $\gamma_i = 0$. У цьому випадку $f(C^i \cdot \mathbf{x}) = 0$, і з огляду на $g_1 \in \langle f \rangle = \mathrm{Ann}(f \oplus 1)$ маємо $g_1(C^i \cdot \mathbf{x}) = 0$.

P.S. Якби вдалось побудувати базис ідеалу $\langle f \oplus 1 \rangle$ і знайти в ньому функцію, що $g' \in \langle f \oplus 1 \rangle = \mathrm{Ann}(f)$, то аналогічно $g' \cdot f = 0$, і рівняння $g'(C^i \cdot \mathbf{x}) = 0$ можна було б записати для тактів, де $\gamma_i = 1$. У такому разі для побудови системи використовувались би *усі* $N = 50000$ тактів. Однак і 25108 нульових тактів значно перевищують необхідну кількість $M = 2081$, тому і одного ідеалу цілком достатньо.

Стан регістру в момент i обчислюється символьно через рекурентне співвідношення ЛРЗЗ, що є значно ефективнішим в порівнянні з безпосереднім множенням супровідної матриці C^i. Усього побудовано **25108** нетривіальних рівнянь степеня 2. Час побудови системи: $\approx 17{,}4$ с.

Перші 10 рівнянь.

1. $x_{62} \cdot x_{22} \oplus x_{62} = 0$
2. $x_{24} \cdot x_0 \oplus x_{24} \cdot x_4 \oplus x_{24} \cdot x_6 \oplus x_{24} \cdot x_9 \oplus ... = 0$
3. $x_{26} \cdot x_1 \oplus x_{26} \cdot x_2 \oplus x_{26} \cdot x_5 \oplus x_{26} \cdot x_6 \oplus ... = 0$
4. $x_{32} \cdot x_0 \oplus x_{32} \cdot x_1 \oplus x_{32} \cdot x_2 \oplus x_{32} \cdot x_3 \oplus ... = 0$
5. $x_{33} \cdot x_0 \oplus x_{33} \cdot x_1 \oplus x_{33} \cdot x_2 \oplus x_{33} \cdot x_3 \oplus ... = 0$
6. $x_{36} \cdot x_0 \oplus x_{36} \cdot x_1 \oplus x_{36} \cdot x_2 \oplus x_{36} \cdot x_3 \oplus ... = 0$
7. $x_{38} \cdot x_0 \oplus x_{38} \cdot x_1 \oplus x_{38} \cdot x_2 \oplus x_{38} \cdot x_3 \oplus ... = 0$
8. $x_{40} \cdot x_0 \oplus x_{40} \cdot x_2 \oplus x_{40} \cdot x_3 \oplus x_{40} \cdot x_5 \oplus ... = 0$
9. $x_{42} \cdot x_0 \oplus x_{42} \cdot x_1 \oplus x_{42} \cdot x_2 \oplus x_{42} \cdot x_6 \oplus ... = 0$
10. $x_{43} \cdot x_0 \oplus x_{43} \cdot x_1 \oplus x_{43} \cdot x_2 \oplus x_{43} \cdot x_3 \oplus ... = 0$

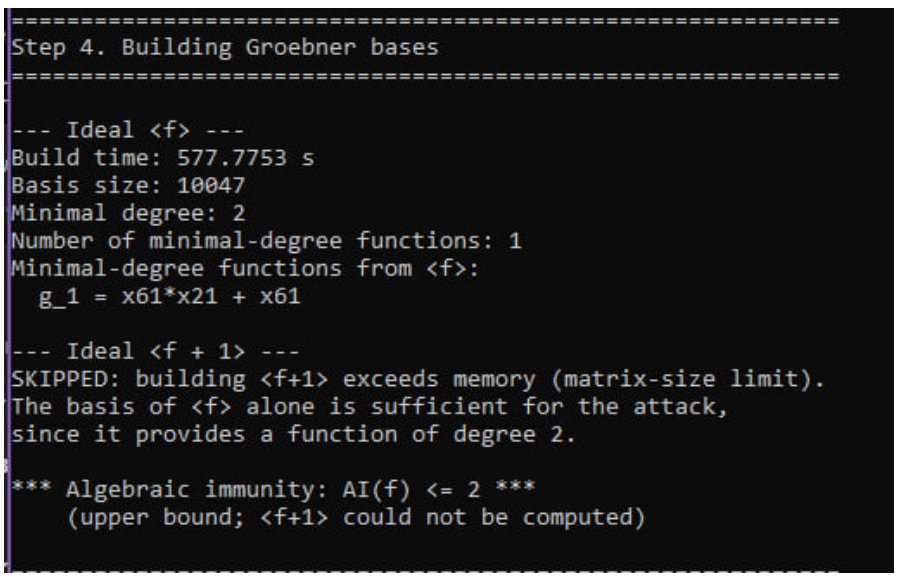


```
(sage) grusevd@grusevdDavid:/mnt/d/Programing2025/CryptoAnalysis_v2/LabWork1$ sage attack.sage
============================================================
Step 1. Initializing Boolean polynomial ring B_64
============================================================
Defining x0, x1, x2, x3, x4, x5, x6, x7, x8, x9, x10, x11, x12, x13, x14, x15, x16, x17, x18, x19, x20, x21, x22, x23, x24, x25, x26, x27, x28, x29
, x30, x31, x32, x33, x34, x35, x36, x37, x38, x39, x40, x41, x42, x43, x44, x45, x46, x47, x48, x49, x50, x51, x52, x53, x54, x55, x56, x57, x58,
x59, x60, x61, x62, x63
Ring: Boolean PolynomialRing in x0, x1, x2, x3, x4, x5, x6, x7, x8, x9, x10, x11, x12, x13, x14, x15, x16, x17, x18, x19, x20, x21, x22, x23, x24,
x25, x26, x27, x28, x29, x30, x31, x32, x33, x34, x35, x36, x37, x38, x39, x40, x41, x42, x43, x44, x45, x46, x47, x48, x49, x50, x51, x52, x53, x5
4, x55, x56, x57, x58, x59, x60, x61, x62, x63
Number of variables: 64

============================================================
Step 2. Building the LFSR companion matrix
============================================================
Feedback polynomial: t^64 + t^63 + t^62 + t^60 + t^59 + t^58 + t^57 + t^56 + t^53 + t^50 + t^47 + t^45 + t^44 + t^43 + t^42 + t^41 + t^40 + t^39 +
t^38 + t^37 + t^36 + t^34 + t^32 + t^30 + t^28 + t^24 + t^18 + t^15 + t^14 + t^13 + t^11 + t^9 + t^6 + t^4 + 1
Polynomial degree: 64
Nonzero feedback coefficient positions: [0, 4, 6, 9, 11, 13, 14, 15, 18, 24, 28, 30, 32, 34, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 47, 50, 53, 56
, 57, 58, 59, 60, 62, 63]

============================================================
Step 3. Filter function
============================================================
Degree of f: 20
Number of monomials in f: 39
  Degree 1: 1 monomials
  Degree 2: 3 monomials
  Degree 3: 3 monomials
  Degree 4: 2 monomials
  Degree 5: 2 monomials
  Degree 6: 3 monomials
  Degree 7: 3 monomials
  Degree 8: 2 monomials
  Degree 9: 2 monomials
  Degree 10: 1 monomials
  Degree 11: 2 monomials
  Degree 12: 1 monomials
  Degree 13: 1 monomials
  Degree 14: 2 monomials
  Degree 15: 2 monomials
  Degree 16: 3 monomials
  Degree 17: 2 monomials
  Degree 18: 2 monomials
  Degree 19: 1 monomials
  Degree 20: 1 monomials
```

Рис. 1.2: Кроки 1–3: ініціалізація кільця B_{64}, супровідна матриця, парсинг f



```
============================================================
Step 4. Building Groebner bases
============================================================

--- Ideal <f> ---
Build time: 577.7753 s
Basis size: 10047
Minimal degree: 2
Number of minimal-degree functions: 1
Minimal-degree functions from <f>:
  g_1 = x61*x21 + x61

--- Ideal <f + 1> ---
SKIPPED: building <f+1> exceeds memory (matrix-size limit).
The basis of <f> alone is sufficient for the attack,
since it provides a function of degree 2.

*** Algebraic immunity: AI(f) <= 2 ***
    (upper bound; <f+1> could not be computed)
```

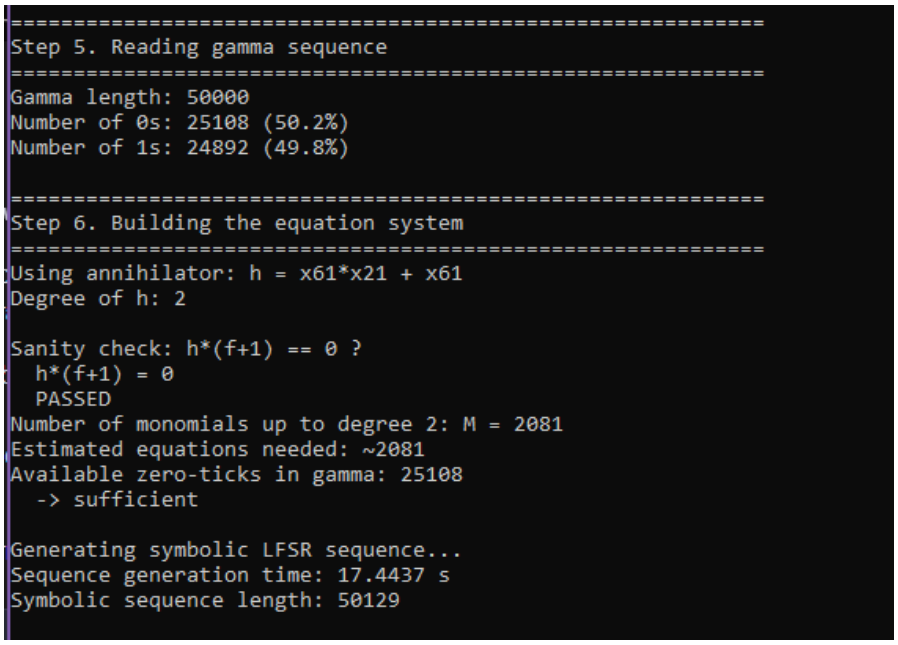
Рис. 1.3: Крок 4: побудова базису Грьобнера ідеалу $\langle f \rangle$

1.2.3 Розв'язання системи

Систему розв'язуємо шляхом побудови базису Грьобнера (алгорит Бухбергера) – об'єднаного ідеалу всіх рівнянь у кільці B_{64}. Редукований базис Грьобнера має 63 елементи виду $x_i \oplus x_{62}$ або просто мономи x_i, що означає наявність однієї вільної змінної (в нашому випадку x_{62}) і, відповідно, двох розв'язків: нульового та ненульового.

Час розв'язання: $\approx 25{,}4$ с. Обидва розв'язки знайдено за допомогою функції `variety()` ($\approx 1{,}1$ с).

Зауваження:.
Як я для себе зрозумів – базис Грьобнера це аналог методу Гауса для поліноміальних систем. Для лінійних систем він буде еквівалентний Гаусу, для нелінійних – узагальнюватиме його (оскільки Гаус не працює з квадратичними рівняннями).



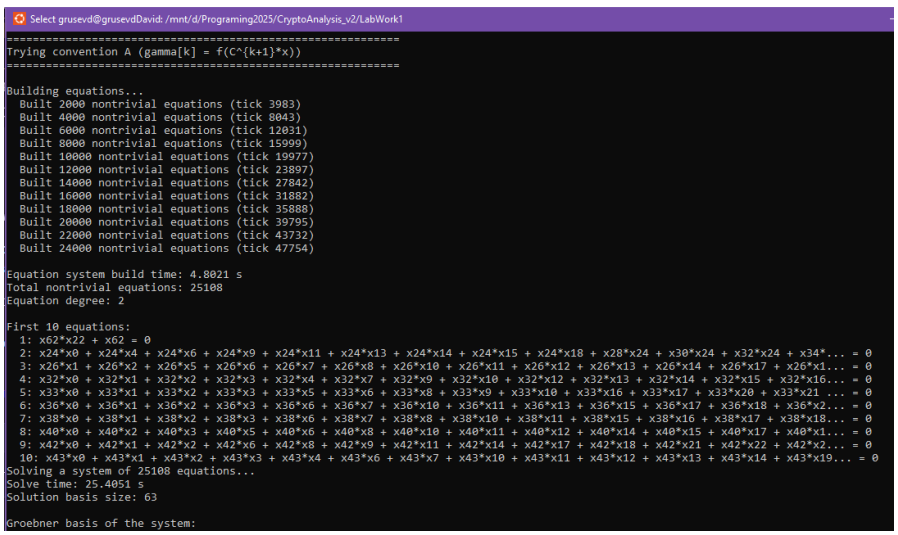
```
============================================================
Step 5. Reading gamma sequence
============================================================
Gamma length: 50000
Number of 0s: 25108 (50.2%)
Number of 1s: 24892 (49.8%)

============================================================
Step 6. Building the equation system
============================================================
Using annihilator: h = x61*x21 + x61
Degree of h: 2

Sanity check: h*(f+1) == 0 ?
  h*(f+1) = 0
  PASSED
Number of monomials up to degree 2: M = 2081
Estimated equations needed: ~2081
Available zero-ticks in gamma: 25108
  -> sufficient

Generating symbolic LFSR sequence...
Sequence generation time: 17.4437 s
Symbolic sequence length: 50129
```

Рис. 1.4: Крок 5: зчитування гами



```
Select grusevd@grusevdDavid: /mnt/d/Programing2025/CryptoAnalysis_v2/LabWork1
============================================================
Trying convention A (gamma[k] = f(C^{k+1}*x))
============================================================

Building equations...
  Built 2000 nontrivial equations (tick 3983)
  Built 4000 nontrivial equations (tick 8043)
  Built 6000 nontrivial equations (tick 12031)
  Built 8000 nontrivial equations (tick 15999)
  Built 10000 nontrivial equations (tick 19977)
  Built 12000 nontrivial equations (tick 23897)
  Built 14000 nontrivial equations (tick 27842)
  Built 16000 nontrivial equations (tick 31882)
  Built 18000 nontrivial equations (tick 35888)
  Built 20000 nontrivial equations (tick 39795)
  Built 22000 nontrivial equations (tick 43732)
  Built 24000 nontrivial equations (tick 47754)

Equation system build time: 4.8021 s
Total nontrivial equations: 25108
Equation degree: 2

First 10 equations:
  1: x62*x22 + x62 = 0
  2: x24*x0 + x24*x4 + x24*x6 + x24*x9 + x24*x11 + x24*x13 + x24*x14 + x24*x15 + x24*x18 + x28*x24 + x30*x24 + x32*x24 + x34*... = 0
  3: x26*x1 + x26*x2 + x26*x5 + x26*x6 + x26*x7 + x26*x8 + x26*x10 + x26*x11 + x26*x12 + x26*x13 + x26*x14 + x26*x17 + x26*x1... = 0
  4: x32*x0 + x32*x1 + x32*x2 + x32*x3 + x32*x4 + x32*x7 + x32*x9 + x32*x10 + x32*x12 + x32*x13 + x32*x14 + x32*x15 + x32*x16... = 0
  5: x33*x0 + x33*x1 + x33*x2 + x33*x3 + x33*x5 + x33*x6 + x33*x8 + x33*x9 + x33*x10 + x33*x16 + x33*x17 + x33*x20 + x33*x21 ... = 0
  6: x36*x0 + x36*x1 + x36*x2 + x36*x3 + x36*x6 + x36*x7 + x36*x10 + x36*x11 + x36*x13 + x36*x15 + x36*x17 + x36*x18 + x36*x2... = 0
  7: x38*x0 + x38*x1 + x38*x2 + x38*x3 + x38*x6 + x38*x7 + x38*x8 + x38*x10 + x38*x11 + x38*x15 + x38*x16 + x38*x17 + x38*x18... = 0
  8: x40*x0 + x40*x2 + x40*x3 + x40*x5 + x40*x6 + x40*x8 + x40*x10 + x40*x11 + x40*x12 + x40*x14 + x40*x15 + x40*x17 + x40*x1... = 0
  9: x42*x0 + x42*x1 + x42*x2 + x42*x6 + x42*x8 + x42*x9 + x42*x11 + x42*x14 + x42*x17 + x42*x18 + x42*x21 + x42*x22 + x42*x2... = 0
  10: x43*x0 + x43*x1 + x43*x2 + x43*x3 + x43*x4 + x43*x6 + x43*x7 + x43*x10 + x43*x11 + x43*x12 + x43*x13 + x43*x14 + x43*x19... = 0
Solving a system of 25108 equations...
Solve time: 25.4051 s
Solution basis size: 63

Groebner basis of the system:
```

Рис. 1.5: Крок 6: побудова системи рівнянь

Знайдений початковий стан: Нульовий розв'язок відкидається за умовою задачі. Єдиний ненульовий розв'язок який маємо, це:

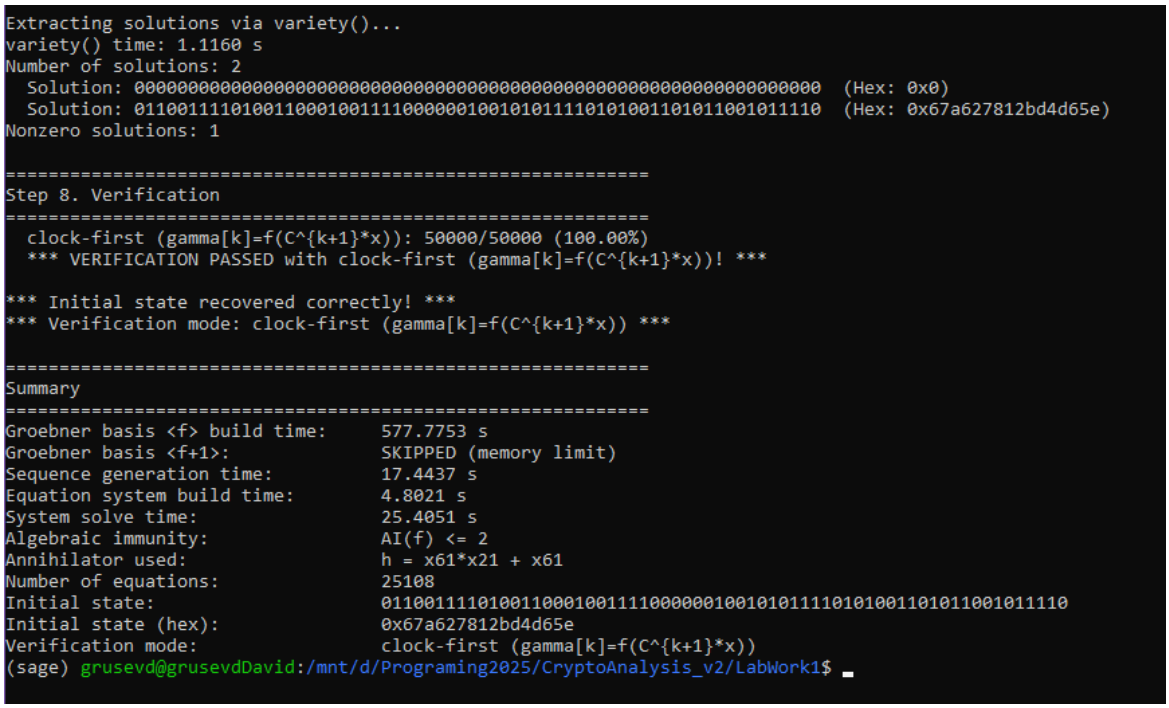
$$\mathbf{x} = (0,1,1,0,0,1,1,1,1,0,1,0,0,1,1,0,0,0,1,0,0,1,1,1,1,0,0,0,0,0,0,1,$$
$$0,0,1,0,1,0,1,1,1,1,0,1,0,1,0,0,1,1,0,1,0,1,1,0,0,1,0,1,1,1,1,0)$$

Якщо перевести binary to hex: `0x67a627812bd4d65e`.

1.2.4 Перевірка

Зі знайденого початкового стану згенеровано $N = 50000$ бітів гами шляхом послідовного тактування ЛРЗЗ та обчислення значень фільтрувальної функції ($\gamma_k = f(C^{k+1} \cdot \mathbf{x})$).

Маємо, що усі 50000 з 50000 бітів збіглися з вхідними даними (**100%**) — ура!)



```
Extracting solutions via variety()...
variety() time: 1.1160 s
Number of solutions: 2
  Solution: 0000000000000000000000000000000000000000000000000000000000000000  (Hex: 0x0)
  Solution: 0110011110100110001001111000000100101011110101001101011001011110  (Hex: 0x67a627812bd4d65e)
Nonzero solutions: 1

============================================================
Step 8. Verification
============================================================
  clock-first (gamma[k]=f(C^{k+1}*x)): 50000/50000 (100.00%)
  *** VERIFICATION PASSED with clock-first (gamma[k]=f(C^{k+1}*x))! ***

*** Initial state recovered correctly! ***
*** Verification mode: clock-first (gamma[k]=f(C^{k+1}*x)) ***

============================================================
Summary
============================================================
Groebner basis <f> build time:     577.7753 s
Groebner basis <f+1>:              SKIPPED (memory limit)
Sequence generation time:          17.4437 s
Equation system build time:        4.8021 s
System solve time:                 25.4051 s
Algebraic immunity:                AI(f) <= 2
Annihilator used:                  h = x61*x21 + x61
Number of equations:               25108
Initial state:                     0110011110100110001001111000000100101011110101001101011001011110
Initial state (hex):               0x67a627812bd4d65e
Verification mode:                 clock-first (gamma[k]=f(C^{k+1}*x))
(sage) grusevd@grusevdDavid:/mnt/d/Programing2025/CryptoAnalysis_v2/LabWork1$
```

Рис. 1.6: Кроки 7-8: розв'язання системи рівнянь, верифікація та підсумок

1.3 Висновки

1. Побудовали редукований базис Грьобнера для ідеалу $\langle f \rangle$ у кільці B_{64} з мономіальним впорядкуванням `degrevlex` (час ≈ 578 с, потужність базису складає 10047). Побудова базису $\langle f \oplus 1 \rangle$ виявилась неможливою через бібліотечне обмеження пам'яті, однак базис $\langle f \rangle$ виявився цілком достатнім для атаки.

2. Визначили верхню межу алгебраїчної імунності: $\mathrm{AI}(f) \leq 2$, що є надзвичайно низьким значенням порівняно з оптимальним $\lceil 64/2 \rceil = 32$.

3. Побудовали систему з 25108 квадратичних рівнянь і розв'язали її за допомогою базису Грьобнера. Вона має два розв'язки: нульовий (відкидається за умовою) та ненульовий $\mathbf{x} =$ `0x67a627812bd4d65e`.

4. Початковий стан перевірено шляхом генерування гами: всі $N = 50000$ бітів збіглись з вхідними даними.

Отже, алгебраїчна атака є ефективним методом відновлення початкового стану фільтрувального генератора, що має низьку алгебраїчну імунність. Загальний час атаки (без врахування побудови базису Грьобнера) становить менше 30 с, що свідчить про повну криптографічну нестійкість заданої фільтрувальної функції.

1.4 Code listing section

Лістинг 1.1: Алгебраїчна атака на генератор Gamma з використанням SAGE

```
#!/usr/bin/env sage
# -*- coding: utf-8 -*-
"""
Algebraic attack on a filtering gamma generator
```

```
"""

import time
import re
import sys

# ============================================================
# Step 1. Define the Boolean polynomial ring B_64
# ============================================================
print("=" * 60)
print("Step 1. Initializing Boolean polynomial ring B_64")
print("=" * 60)

n = 64
B = BooleanPolynomialRing(n, 'x', order='degrevlex')
B.inject_variables()
x = B.gens()
print(f"Ring: {B}")
print(f"Number of variables: {n}")

# ============================================================
# Step 2. Feedback polynomial and companion matrix
# ============================================================
print("\n" + "=" * 60)
print("Step 2. Building the LFSR companion matrix")
print("=" * 60)

# p(x) = x^64 + x^63 + x^62 + x^60 + x^59 + x^58 + x^57 + x^56 +
#         x^53 + x^50 + x^47 + x^45 + x^44 + x^43 + x^42 + x^41 +
#         x^40 + x^39 + x^38 + x^37 + x^36 + x^34 + x^32 + x^30 +
#         x^28 + x^24 + x^18 + x^15 + x^14 + x^13 + x^11 + x^9 +
#         x^6 + x^4 + 1

R_poly = GF(2)['t']
t = R_poly.gen()
p = (t^64 + t^63 + t^62 + t^60 + t^59 + t^58 + t^57 + t^56 +
     t^53 + t^50 + t^47 + t^45 + t^44 + t^43 + t^42 + t^41 +
     t^40 + t^39 + t^38 + t^37 + t^36 + t^34 + t^32 + t^30 +
     t^28 + t^24 + t^18 + t^15 + t^14 + t^13 + t^11 + t^9 +
     t^6 + t^4 + 1)

print(f"Feedback polynomial: {p}")
print(f"Polynomial degree: {p.degree()}")

# Companion matrix over GF(2)
C_gf2 = companion_matrix(p, format='bottom')

# Feedback coefficients (c_0, c_1, ..., c_63)
# Recurrence: s_{j+64} = c_0*s_j + c_1*s_{j+1} + ... + c_63*s_{j+63}
feedback_coeffs = [int(p[i]) for i in range(64)]
feedback_indices = [i for i in range(64) if feedback_coeffs[i] == 1]
```

```python
print(f"Nonzero feedback coefficient positions: {feedback_indices}")

# ============================================================
# Step 3. Read and parse the filter function
# ============================================================
print("\n" + "=" * 60)
print("Step 3. Filter function")
print("=" * 60)

with open('filterfuncV10.txt', 'r') as file:
    func_str = file.read().strip()

# Parse LaTeX notation: x_{i} -> x[i], space = multiplication, " + " = addition
terms = func_str.split(' + ')
f = B(0)
for term in terms:
    var_indices = re.findall(r'x_\{(\d+)\}', term)
    monomial = B(1)
    for v in var_indices:
        monomial *= x[int(v)]
    f += monomial

print(f"Degree of f: {f.degree()}")
print(f"Number of monomials in f: {len(f)}")

from collections import Counter
deg_distribution = Counter()
for monom in f:
    deg_distribution[monom.degree()] += 1
for d in sorted(deg_distribution.keys()):
    print(f"  Degree {d}: {deg_distribution[d]} monomials")

# ============================================================
# Step 4. Build Groebner bases for ideals <f> and <f+1>
# ============================================================
print("\n" + "=" * 60)
print("Step 4. Building Groebner bases")
print("=" * 60)

# --- Groebner basis of ideal <f> ---
print("\n--- Ideal <f> ---")
t0 = time.time()
I_f = B.ideal(f)
GB_f = I_f.groebner_basis()
t_gb_f = time.time() - t0
print(f"Build time: {t_gb_f:.4f} s")
print(f"Basis size: {len(GB_f)}")

degrees_f = [g.degree() for g in GB_f]
min_deg_f = min(degrees_f)
```

```python
min_funcs_f = [g for g in GB_f if g.degree() == min_deg_f]
print(f"Minimal degree: {min_deg_f}")
print(f"Number of minimal-degree functions: {len(min_funcs_f)}")
print("Minimal-degree functions from <f>:")
for i, g in enumerate(min_funcs_f):
    print(f"  g_{i+1} = {g}")

# --- Groebner basis of ideal <f+1> ---
# For our variant, building <f+1> exceeds memory limits
# (polybori throws "Built-in matrix-size exceeded!").
# As stated in the task: if building one basis is too resource-intensive,
# we build only the other - provided it suffices for the attack.
# Since <f> yields a degree-2 function, this is more than sufficient.

print("\n--- Ideal <f + 1> ---")
print("SKIPPED: building <f+1> exceeds memory (matrix-size limit).")
print("The basis of <f> alone is sufficient for the attack,")
print("since it provides a function of degree 2.")
GB_f1 = None
t_gb_f1 = None
min_deg_f1 = float('inf')
min_funcs_f1 = []

# Algebraic immunity (upper bound)
# True AI = min(min_deg in <f>, min_deg in <f+1>)
# We only know the <f> side, so AI(f) <= min_deg_f
print(f"\n*** Algebraic immunity: AI(f) <= {min_deg_f} ***")
print(f"    (upper bound; <f+1> could not be computed)")

e = min_deg_f  # degree of equations we will use

# ============================================================
# Step 5. Read the gamma sequence
# ============================================================
print("\n" + "=" * 60)
print("Step 5. Reading gamma sequence")
print("=" * 60)

with open('gammaV10.txt', 'r') as file:
    gamma_str = file.read().strip()
gamma = [int(b) for b in gamma_str]
N = len(gamma)
n_zeros = gamma.count(0)
n_ones = gamma.count(1)
print(f"Gamma length: {N}")
print(f"Number of 0s: {n_zeros} ({float(100*n_zeros/N):.1f}%)")
print(f"Number of 1s: {n_ones} ({float(100*n_ones/N):.1f}%)")

# ============================================================
# Step 6. Build the system of lower-degree equations
# ============================================================
```

```
print("\n" + "=" * 60)
print("Step 6. Building the equation system")
print("=" * 60)

# Attack strategy:
#   We have h in <f> with deg h = 2.
#   <f> = Ann(f + 1), so h * (f + 1) = 0.
#   When gamma_i = 0  =>  f(C^i * x) = 0  =>  h(C^i * x) = 0.
#   We use ONLY the ticks where gamma_i = 0.

h = min_funcs_f[0]  # the degree-2 annihilator
print(f"Using annihilator: h = {h}")
print(f"Degree of h: {h.degree()}")

# Sanity check: h must satisfy h*(f+1) = 0 in B_64
print("\nSanity check: h*(f+1) == 0 ?")
check = h * (f + 1)
print(f"  h*(f+1) = {check}")
assert check == 0, "FATAL: annihilator check failed!"
print("  PASSED")

# Number of monomials up to degree e in n variables (linearization bound)
M = sum(binomial(n, k) for k in range(e + 1))
print(f"Number of monomials up to degree {e}: M = {M}")
print(f"Estimated equations needed: ~{M}")
print(f"Available zero-ticks in gamma: {n_zeros}")
print(f"  -> {'sufficient' if n_zeros >= M else 'INSUFFICIENT!'}")

# Generate the symbolic LFSR sequence via the recurrence relation.
# s[j] is a linear combination of x_0,...,x_{n-1} in B.
# The LFSR state at time i is: (s[i], s[i+1], ..., s[i+n-1]) = C^i * x
#
# Convention from the theory: gamma_i = f(C^i * x), i = 1,2,3,...
# In the file: gamma[0] = gamma_1, gamma[1] = gamma_2, etc.
# So gamma[k] corresponds to state (s[k+1], ..., s[k+64]).
#
# However, the exact convention is uncertain, so we will try BOTH:
#   Convention A: gamma[k] = f(C^{k+1} * x)  =>  state s[k+1 : k+65]
#   Convention B: gamma[k] = f(C^{k}   * x)  =>  state s[k   : k+64]

print("\nGenerating symbolic LFSR sequence...")
t0 = time.time()

max_ticks = N  # use ALL gamma bits

# s[0] = x_0, ..., s[63] = x_63 (initial state variables)
s = list(x)

# Extend via LFSR recurrence: s[j+64] = sum c_i * s[j+i]
for j in range(max_ticks + n + 1):
    new_bit = B(0)
```

```
    base = len(s) - n
    for idx in feedback_indices:
        new_bit += s[base + idx]
    s.append(new_bit)

t_seq = time.time() - t0
print(f"Sequence generation time: {t_seq:.4f} s")
print(f"Symbolic sequence length: {len(s)}")

# Build equations using Convention A (theory: gamma[k] <-> C^{k+1})
# If this doesn't verify, we will try Convention B.
for convention_name, offset in [("A (gamma[k] = f(C^{k+1}*x))", 1),
                                ("B (gamma[k] = f(C^{k}*x))", 0)]:
    print(f"\n{'='*60}")
    print(f"Trying convention {convention_name}")
    print(f"{'='*60}")

    print("\nBuilding equations...")
    t0 = time.time()

    equations = []
    eq_count = 0

    for k in range(max_ticks):
        if gamma[k] != 0:
            continue  # only use ticks where gamma = 0

        i = k + offset  # state index
        state_i = s[i:i+n]
        eq = h(*state_i)

        if eq == 0:
            continue
        if eq == 1:
            print(f"  WARNING: contradiction at tick k={k}!")
            continue

        equations.append(eq)
        eq_count += 1

        if eq_count % 2000 == 0:
            print(f"  Built {eq_count} nontrivial equations (tick {k})")

    t_build = time.time() - t0
    print(f"\nEquation system build time: {t_build:.4f} s")
    print(f"Total nontrivial equations: {len(equations)}")
    print(f"Equation degree: {e}")

    # Print the first 10 equations
    print("\nFirst 10 equations:")
    for i_eq, eq in enumerate(equations[:10]):
```

```
        eq_str = str(eq)
        if len(eq_str) > 120:
            eq_str = eq_str[:120] + "..."
        print(f"  {i_eq+1}: {eq_str} = 0")

    # ============================================================
    # Step 7. Solve the equation system
    # ============================================================
    print(f"\nSolving a system of {len(equations)} equations...")
    t0 = time.time()
    sol_ideal = B.ideal(equations)
    sol_GB = sol_ideal.groebner_basis()
    t_solve = time.time() - t0
    print(f"Solve time: {t_solve:.4f} s")
    print(f"Solution basis size: {len(sol_GB)}")

    # Print the basis
    print("\nGroebner basis of the system:")
    for g in sol_GB:
        print(f"  {g}")

    # ============================================================
    # Extract solutions using variety()
    # ============================================================
    print("\nExtracting solutions via variety()...")
    t0 = time.time()
    try:
        V = sol_ideal.variety()
    except Exception as ex:
        print(f"variety() failed: {ex}")
        V = []
    t_var = time.time() - t0
    print(f"variety() time: {t_var:.4f} s")
    print(f"Number of solutions: {len(V)}")

    if len(V) == 0:
        print("No solutions found, trying next convention...")
        continue

    # Convert to bit vectors
    solutions = []
    for sol_dict in V:
        sol = [int(sol_dict[x[i]]) for i in range(n)]
        solutions.append(sol)
        sol_str = ''.join(map(str, sol))
        print(f"  Solution: {sol_str}  (Hex: {hex(int(sol_str, 2))})")

    # Filter nonzero (task requires nonzero initial state)
    nonzero_solutions = [s_sol for s_sol in solutions if any(b == 1 for b in
       s_sol)]
    print(f"Nonzero solutions: {len(nonzero_solutions)}")
```

```python

    if len(nonzero_solutions) == 0:
        print("Only zero solution found, trying next convention...")
        continue

    # ============================================================
    # Step 8. Verification (try BOTH clock orderings)
    # ============================================================
    print(f"\n{'='*60}")
    print("Step 8. Verification")
    print(f"{'='*60}")

    verified_solution = None
    verified_mode = None

    for solution in nonzero_solutions:
        for mode_name, clock_first in [("clock-first (gamma[k]=f(C^{k+1}*x))", True),
                                       ("eval-first  (gamma[k]=f(C^{k}*x))", False)]:
            curr = vector(GF(2), solution)

            generated_gamma = []
            for i_tick in range(N):
                if clock_first:
                    curr = C_gf2 * curr
                    val = f(*list(curr))
                else:
                    val = f(*list(curr))
                    curr = C_gf2 * curr
                generated_gamma.append(int(val))

            match_count = sum(1 for i_m in range(N)
                              if gamma[i_m] == generated_gamma[i_m])
            pct = float(100 * match_count / N)
            print(f"  {mode_name}: {match_count}/{N} ({pct:.2f}%)")

            if match_count == N:
                print(f"  *** VERIFICATION PASSED with {mode_name}! ***")
                verified_solution = solution
                verified_mode = mode_name
                break

        if verified_solution is not None:
            break

    if verified_solution is not None:
        print(f"\n*** Initial state recovered correctly! ***")
        print(f"*** Verification mode: {verified_mode} ***")
        break  # exit the convention loop
    else:
```

```python
        print(f"\nConvention {convention_name} did not produce a valid solution.")

# ============================================================
# Summary
# ============================================================
print("\n" + "=" * 60)
print("Summary")
print("=" * 60)
print(f"Groebner basis <f> build time:     {t_gb_f:.4f} s")
print(f"Groebner basis <f+1>:              SKIPPED (memory limit)")
print(f"Sequence generation time:          {t_seq:.4f} s")
print(f"Equation system build time:        {t_build:.4f} s")
print(f"System solve time:                 {t_solve:.4f} s")
print(f"Algebraic immunity:                AI(f) <= {e}")
print(f"Annihilator used:                  h = {h}")
print(f"Number of equations:               {len(equations)}")
if verified_solution is not None:
    vs = ''.join(map(str, verified_solution))
    print(f"Initial state:                     {vs}")
    print(f"Initial state (hex):               {hex(int(vs, 2))}")
    print(f"Verification mode:                 {verified_mode}")
else:
    print("Initial state:                     NOT FOUND")
```

Список використаних джерел

[1] Nicolas T. Courtois and Willi Meier. «Algebraic Attacks on Stream Ciphers with Linear Feedback». In: *Advances in Cryptology — EUROCRYPT 2003*. Vol. 2656. Lecture Notes in Computer Science. Springer, 2003, pp. 345–359.